

Chapter 3: Python Programming Essentials_part 02

This part 2 covers Object-Oriented Programming (OOP) with magic methods, generators and iterators, and exception handling.

10. Object-Oriented Programming (OOP) in Python

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of **objects**, which contain **data** (attributes) and **behavior** (methods).

Main OOP Concepts

- **Class:** A blueprint (model) for creating objects.
- **Object (Instance):** A specific instance of a class.
- **Attribute:** A variable that belongs to an object.
- **Method:** A function that belongs to a class.

10.2. Defining a Class and Creating Objects

```
In [1]: class Person:
        # Constructor
        def __init__(self, name, age):
            self.name = name
            self.age = age

        # Method
        def greet(self):
            print(f"Hello, my name is {self.name} and I am {self.age} years old.")

        # Creating objects
        p1 = Person("Ali", 25)
        p2 = Person("Ahmed", 30)

        p1.greet()
        p2.greet()
```

Hello, my name is Ali and I am 25 years old.

Hello, my name is Ahmed and I am 30 years old.

10.3. Class and Instance Attributes

```
In [2]: class Student:
        speciality = "AI" # Class attribute (shared by all instances)
        def __init__(self, name, age):
            self.name = name # Instance attribute
            self.age = age # Instance attribute

        # Create two Student objects
        student1 = Student("Amine", 20)
```

```

student2 = Student("Sara", 22)

# Access class and instance attributes
print(Student.speciality)           # Access via class
print(student1.name, student1.age, student1.speciality)
print(student2.name, student2.age, student2.speciality)

```

AI

Amine 20 AI

Sara 22 AI

10. 4. Encapsulation (Data Hiding)

Encapsulation restricts direct access to some of an object's data.

```

In [3]: class BankAccount:
        def __init__(self, balance):
            self.__balance = balance # Private attribute

        def deposit(self, amount):
            self.__balance += amount

        def get_balance(self):
            return self.__balance

account = BankAccount(1000)
account.deposit(500)
print(account.get_balance())
# print(account.__balance) # ❌ AttributeError

```

1500

10.5. Inheritance

Inheritance allows a class to use properties and methods from another class.

```

In [4]: class Student:
        def __init__(self, name):
            self.name = name

        def study(self):
            print(f"{self.name} is studying.")

# Subclass for undergraduate students
class UndergraduateStudent(Student):
    def study(self):
        print(f"{self.name} is studying basic AI concepts.")

# Subclass for graduate students
class GraduateStudent(Student):
    def study(self):
        print(f"{self.name} is working on AI research.")

# Create instances
s1 = UndergraduateStudent("Amine")
s2 = GraduateStudent("Sara")
s3 = Student("Yassine")

# Call methods
s1.study()

```

```
s2.study()  
s3.study()
```

Amine is studying basic AI concepts.
Sara is working on AI research.
Yassine is studying.

10.6. Polymorphism

Polymorphism allows different classes to use the same interface with different implementations.

```
In [5]: # Base class (optional in Python, but helps structure)
class Student:
    def study(self):
        pass # Each subclass will implement its own version

# Subclass for undergraduate students
class UndergraduateStudent(Student):
    def __init__(self, name):
        self.name = name

    # Override study() method
    def study(self):
        print(f"{self.name} studies introductory AI courses.")

# Subclass for graduate students
class GraduateStudent(Student):
    def __init__(self, name):
        self.name = name

    # Override study() method
    def study(self):
        print(f"{self.name} conducts AI research and writes papers.")

# Create a list of students of different levels
students = [UndergraduateStudent("Amine"), GraduateStudent("Sara")]

# Polymorphism in action: same method (study) but different implementations
for student in students:
    student.study()
```

Amine studies introductory AI courses.
Sara conducts AI research and writes papers.

10.7. Abstraction

Abstraction hides internal details and shows only the necessary features.

```
In [6]: from abc import ABC, abstractmethod

# Abstract base class
class Student(ABC):
    @abstractmethod
    def get_final_grade(self):
        pass # Each subclass will provide its own implementation

# Concrete subclass for undergraduate students
class UndergraduateStudent(Student):
    def __init__(self, name, midterm, final):
        self.name = name
```

```

        self.midterm = midterm
        self.final = final

    def get_final_grade(self):
        return (self.midterm * 0.4) + (self.final * 0.6)

# Concrete subclass for graduate students
class GraduateStudent(Student):
    def __init__(self, name, research_score, exam_score):
        self.name = name
        self.research_score = research_score
        self.exam_score = exam_score

    def get_final_grade(self):
        return (self.research_score * 0.5) + (self.exam_score * 0.5)

# Create objects and display results
s1 = UndergraduateStudent("Amine", 14, 16)
s2 = GraduateStudent("Sara", 18, 15)

print(f"{s1.name}'s final grade: {s1.get_final_grade()}")
print(f"{s2.name}'s final grade: {s2.get_final_grade()}")

```

Amine's final grade: 15.2
 Sara's final grade: 16.5

10.8. Magic Methods (init, str, etc.)

Magic methods (also called dunder methods) define special behaviors of objects.

```

In [7]: # Define a Student class
class Student:
    # __init__ is the constructor: called automatically when creating an object
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade # numeric grade (e.g., 0-20)

    # __str__ defines how the object appears when printed
    def __str__(self):
        return f"Student: {self.name}, Grade: {self.grade}"

    # __add__ defines the behavior of the + operator between two Student objects
    def __add__(self, other):
        # Returns a new Student with the average of the two grades
        avg_grade = (self.grade + other.grade) / 2
        return Student(f"{self.name} & {other.name}", avg_grade)

# Create two students
s1 = Student("Amine", 15)
s2 = Student("Sara", 17)

# Use + operator between two students (calls __add__)
group = s1 + s2

# Print the result (calls __str__)
print(group) # Output: Student: Amine & Sara, Grade: 16.0

```

Student: Amine & Sara, Grade: 16.0

Python Magic Methods Examples

Magic Method	Rôle / Quand Python l'appelle	Exemple / Usage
<code>__init__(self, name, grade)</code>	Constructeur : appelé lors de la création d'un étudiant	<code>s = Student("Amel", 17)</code>
<code>__str__(self)</code>	Retourne une chaîne lisible pour <code>print(obj)</code>	<code>print(s)</code> → Student: Amel, Grade: 17
<code>__repr__(self)</code>	Représentation officielle (souvent utilisée en débogage)	<code>repr(s)</code> → Student('Amel', 17)
<code>__add__(self, other)</code>	Définit le comportement de <code>+</code> entre deux étudiants (ex: moyenne de leurs notes)	<code>s1 + s2</code> → moyenne des notes
<code>__lt__(self, other)</code>	Définit le comportement de <code><</code> (comparaison de notes)	<code>s1 < s2</code> → <code>True</code> si la note de <code>s1</code> est inférieure
<code>__le__(self, other)</code>	Définit le comportement de <code><=</code>	<code>s1 <= s2</code>
<code>__gt__(self, other)</code>	Définit le comportement de <code>></code>	<code>s1 > s2</code>
<code>__ge__(self, other)</code>	Définit le comportement de <code>>=</code>	<code>s1 >= s2</code>
<code>__eq__(self, other)</code>	Définit le comportement de <code>==</code> (égalité de notes)	<code>s1 == s2</code>

10.9. Composition

```
In [8]: class StudentCard:
    def __init__(self, card_id):
        self.card_id = card_id

    def show(self):
        print(f"Student Card ID: {self.card_id}")

class Student:
    def __init__(self, name, card_id):
        self.name = name
        self.card = StudentCard(card_id) # Composition: Student HAS a StudentCard

    def show_info(self):
        print(f"Name: {self.name}")
        self.card.show()

# Create a student
s1 = Student("Amine", "AI2025-001")
s1.show_info()
```

Name: Amine
Student Card ID: AI2025-001

10.10. Aggregation

```
In [9]: class Student:
    def __init__(self, name):
        self.name = name

class Teacher:
    def __init__(self, name):
        self.name = name
        self.students = [] # Aggregation: Teacher HAS many Students

    def add_student(self, student):
```

```

        self.students.append(student)

    def show_students(self):
        print(f"Teacher: {self.name}")
        print("Students:")
        for s in self.students:
            print(f" - {s.name}")

# Create students (independent of any teacher)
s1 = Student("Amine")
s2 = Student("Sara")

# Create teacher and associate existing students
t1 = Teacher("Dr. BOUSTIL")
t1.add_student(s1)
t1.add_student(s2)

t1.show_students()

```

Teacher: Dr. BOUSTIL

Students:

- Amine
- Sara

11. Iterators and Generators in Python

Iterators and generators are **advanced Python features** that allow efficient and **lazy** (on-demand) data processing.

They are particularly useful when working with **large datasets** that cannot fit entirely into memory.

11.1. Problem statement

In data science or healthcare projects for example, we often deal with very large CSV files — sometimes millions of patient records.

If we try to load everything at once using a list, the program may become slow or crash due to memory exhaustion.

We need a way to read the file line by line, process each patient, and never keep the whole dataset in memory.

That's where iterators and generators (yield) become essential.

11.2. Iterators

An **iterator** is an object that represents a **stream of data**; it returns elements **one at a time**.

Anything that can be looped over using a `for` loop is called **iterable**.

```

In [10]: ## Basic Example : create an iterator by iter
nums = [1, 2, 3]
it = iter(nums) # Create an iterator from the list
print(it.__next__())# 1, here is the same syntax as print(next(it))

print(next(it)) # 2
print(next(it)) # 3
# print(next(it)) # ❌ StopIteration (no more elements)

```

1
2
3

11.3. Generators

The `yield` keyword is used in a function to make it a **generator**.

A generator produces a sequence of values **one at a time**, instead of returning them all at once.

Unlike `return`, which **ends** a function,

`yield` **pauses** the function and **remembers its state**,

so it can **continue from where it left off** next time.

```
In [11]: ## Example: create an iterator by a generator using yield
def countdown(n):
    while n > 0:
        yield n+4           # Produce one value, then pause
        n -= 1
    print("Done!")

# Using the generator
for number in countdown(3):
    print(number)
    print("yes")
```

7
yes
6
yes
5
yes
Done!

Problem: Processing 1 Million Patients — Memory Explosion

Imagine a file `patients.csv` with 1,000,000 lines like this:

name,age,blood_pressure

Patient_1,30,120

Patient_2,45,155

...

Patient_1000000,50,140

```
In [ ]: ## Bad solution - Loads entire file
import csv

def read_all_patients(filename):
    with open(filename) as f:
        reader = csv.DictReader(f)
        return list(reader) # Loads all rows ❌

patients = read_all_patients("patients.csv")
for p in patients:
    print(p["name"])
```

```
In [ ]: ## Correct solution – use a generator (yield)
import csv

def read_patients(filename):
    with open(filename, newline='') as f:
        reader = csv.DictReader(f)
        for row in reader:
            yield row # returns one patient at a time ✓

for p in read_patients("patients.csv"):
    print(p["name"])
```

12. Error Handling and Exceptions

In data science, errors are common — especially when dealing with

- **missing files,**
- **invalid values** or
- **messy data.**

Handling errors gracefully helps build **robust and reliable data pipelines**.

What Are Exceptions?

Exceptions are **runtime errors** that interrupt the normal flow of a program.

```
In [ ]: # Example:
data = open("patients.csv") # Raises FileNotFoundError if file doesn't exist
```

12.1. Try–Except Blocks

Use try–except to catch and handle exceptions safely.

```
In [12]: # generic except
try:
    data = open("patients.csv")
except :
    print(" Something went wrong")
```

Something went wrong

```
In [13]: # specefic except
try:
    data = open("patients.csv")
except FileNotFoundError:
    print(" Data file not found! Please check the file path.")
```

Data file not found! Please check the file path.

```
In [14]: # Catching Multiple Errors

# You can handle different errors in one block:

try:
    result = int("abc") / 0
except ValueError:
    print("Invalid data type – conversion failed.")
```



```
except ZeroDivisionError:  
    print("Division by zero detected!")
```

Invalid data type – conversion failed.

12.2. Finally and Else

- **else** runs if no exception occurs
- **finally** runs no matter what try:

```
In [15]: try:  
        f = open("patients.csv")  
    except FileNotFoundError:  
        print("File missing.")  
    else:  
        print("File opened successfully!")  
    finally:  
        print("Process completed.")
```

File missing.

Process completed.

Best Practices

- Use specific exceptions instead of a generic except.
- Use finally to release resources.
- Validate user input before processing.